

Homework 06 Solutions

Prerith Arunkumar (prerith2)

Test and training ROC curves for logistic regression and KNN on the Wisconsin breast cancer data, and a peer evaluation of Homework 5.

Use of AI tools. I used Claude (Claude Code) to draft the R code and exposition for Questions 1 and 2 and to run them. Every number below comes from the code output shown in this report, and I checked the numbers quoted in the text against it. Question 3, part 2, was written by me without AI assistance.

```
library(pROC)

dat <- read.csv("data/breast-cancer.csv")
xvars <- c("mean_radius", "mean_texture", "mean_smoothness")

set.seed(432)
train_id <- sample(nrow(dat), round(0.7 * nrow(dat)))
train <- dat[train_id, ]
test <- dat[-train_id, ]

# Standardize with the training means and standard deviations only.
mu <- colMeans(train[, xvars])
s <- apply(train[, xvars], 2, sd)
Xtr <- scale(as.matrix(train[, xvars]), center = mu, scale = s)
Xte <- scale(as.matrix(test[, xvars]), center = mu, scale = s)
ytr <- train$y
yte <- test$y
c(n_train = nrow(Xtr), n_test = nrow(Xte), malignant_train = sum(ytr), malignant_test = sum(yte))
```

n_train	n_test	malignant_train	malignant_test
398	171	147	65

```
# Equal-weight KNN probability estimate of P(y = 1 | x): the fraction of the k nearest
# training points (Euclidean distance) with y = 1. Returns an (n_query x length(ks)) matrix.
knn_prob <- function(Xtr, ytr, Xq, ks) {
  d2 <- outer(rowSums(Xq^2), rowSums(Xtr^2), "+") - 2 * Xq %*% t(Xtr)
  ord <- t(apply(d2, 1, order))
  sapply(ks, function(k) rowMeans(matrix(ytr[ord[, 1:k, drop = FALSE]], nrow = nrow(Xq))))
}
```

Question 1: Compare KNN and logistic regression

Seed 432; 398 training and 171 test cases, the same split used for every model below.

```
fit_lr <- glm(y ~ ., family = binomial,
             data = data.frame(Xtr, y = ytr))
p_lr <- predict(fit_lr, newdata = data.frame(Xte), type = "response")
```

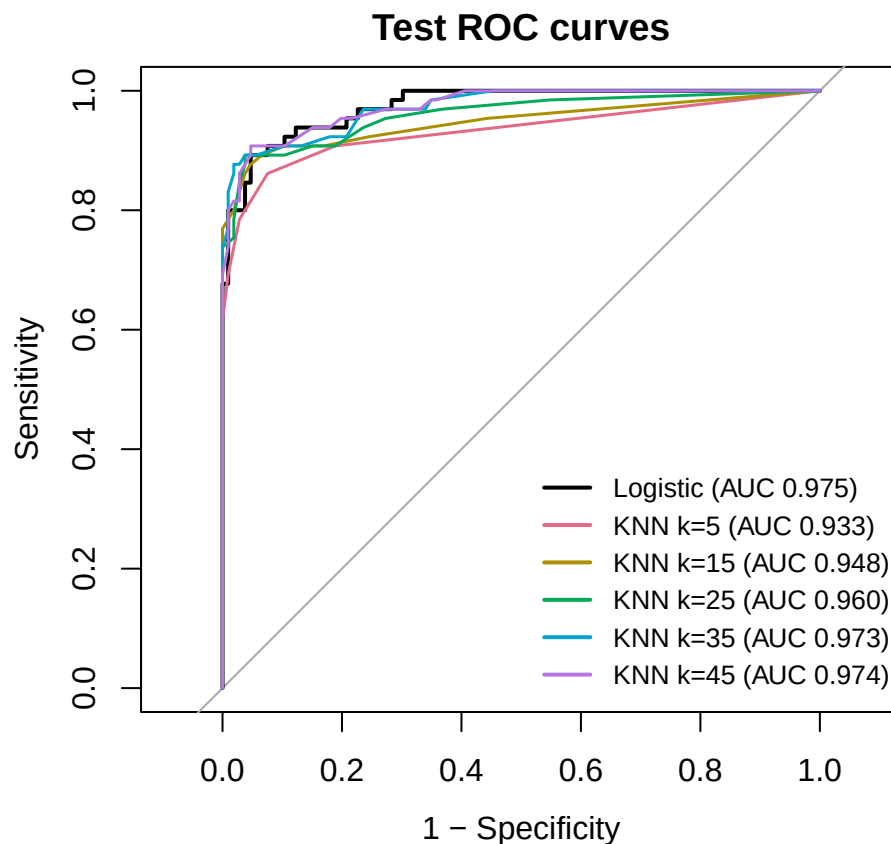
```

ks1 <- c(5, 15, 25, 35, 45)
p_knn <- knn_prob(Xtr, ytr, Xte, ks1)

probs1 <- cbind(Logistic = p_lr, setNames(as.data.frame(p_knn), paste0("KNN k=", ks1)))
rocs1 <- lapply(probs1, function(p) roc(yte, p, levels = c(0, 1), direction = "<", quiet = TRUE))
auc1 <- sapply(rocs1, function(r) as.numeric(auc(r)))

cols <- c("black", hcl.colors(5, "Dark 3"))
plot(rocs1[[1]], col = cols[1], lwd = 2, legacy.axes = TRUE,
      main = "Test ROC curves")
for (j in 2:6) plot(rocs1[[j]], col = cols[j], lwd = 1.5, add = TRUE)
legend("bottomright", sprintf("%s (AUC %.3f)", names(auc1), auc1),
      col = cols, lwd = 2, cex = 0.8, bty = "n")

```



```
round(auc1, 4)
```

Logistic	KNN k=5	KNN k=15	KNN k=25	KNN k=35	KNN k=45
0.9746	0.9329	0.9480	0.9599	0.9727	0.9741

Preferred model. The test AUC values are close: every model is between 0.933 and 0.975. Logistic regression has the highest test AUC (0.975), followed closely by KNN with $k = 45$ (0.974) and $k = 35$ (0.973). For KNN the test AUC increases steadily with k ; $k = 5$ is clearly the weakest (0.933) because with only five neighbors the estimated probability can take only six values (0, 0.2, ..., 1) and is noisy, so its ROC curve is coarse.

I prefer **logistic regression**. It has the highest test AUC, although its margin over KNN with $k = 35$ or 45 is small relative to the sampling variability of an AUC from 171 test cases on a single split. It also has practical advantages: it needs no choice of k , it gives smooth, interpretable probabilities with a coefficient for each standardized predictor,

and it predicts a new case without storing and searching the whole training set. That KNN keeps improving as it averages over larger neighborhoods also suggests the true probability surface is smooth, which a logistic model captures well.

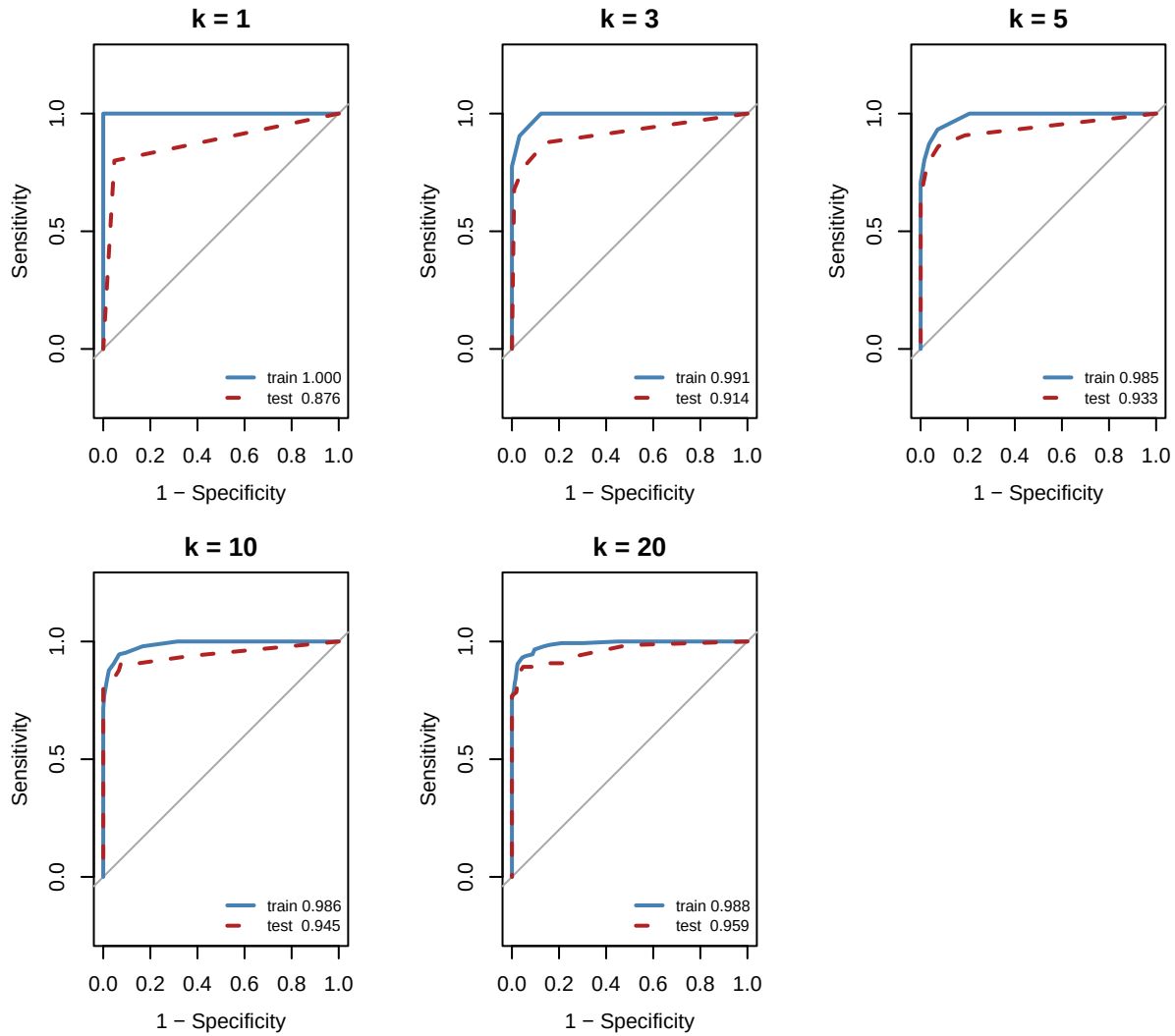
Question 2: Training and test ROC curves

Same split and standardization as Question 1. For the training predictions each training point is its own nearest neighbor (distance 0), so it is included among its k neighbors.

```
ks2 <- c(1, 3, 5, 10, 20)
P_tr <- knn_prob(Xtr, ytr, Xtr, ks2)
P_te <- knn_prob(Xtr, ytr, Xte, ks2)

auc2 <- t(apply(seq_along(ks2), function(j) c(
  k      = ks2[j],
  train = as.numeric(auc(roc(ytr, P_tr[, j], levels = c(0, 1), direction = "<", quiet = TRUE))),
  test  = as.numeric(auc(roc(yte, P_te[, j], levels = c(0, 1), direction = "<", quiet = TRUE))))))

op <- par(mfrow = c(2, 3), mar = c(4, 4, 2.5, 1))
for (j in seq_along(ks2)) {
  r_tr <- roc(ytr, P_tr[, j], levels = c(0, 1), direction = "<", quiet = TRUE)
  r_te <- roc(yte, P_te[, j], levels = c(0, 1), direction = "<", quiet = TRUE)
  plot(r_tr, col = "steelblue", lwd = 2, legacy.axes = TRUE, main = paste0("k = ", ks2[j]))
  plot(r_te, col = "firebrick", lwd = 2, lty = 2, add = TRUE)
  legend("bottomright", c(sprintf("train %.3f", auc2[j, "train"]), sprintf("test  %.3f", auc2[j, "test"])),
        col = c("steelblue", "firebrick"), lty = 1:2, lwd = 2, cex = 0.8, bty = "n")
}
par(op)
```



```
data.frame(round(auc2, 4), gap = round(auc2[, "train"] - auc2[, "test"], 4))
```

	k	train	test	gap
1	1	1.0000	0.8764	0.1236
2	3	0.9905	0.9138	0.0768
3	5	0.9847	0.9329	0.0518
4	10	0.9862	0.9446	0.0415
5	20	0.9880	0.9586	0.0294

Training versus test performance. At every k the training AUC is at least as high as the test AUC, because each training probability is computed from a neighborhood that contains the observation itself (and other nearby training points the model has already “seen”). The gap is largest at $k = 1$ and shrinks steadily as k grows: the training AUC drops from 1.000 to about 0.99 by $k = 3$ and then stays nearly flat (0.988 at $k = 20$), while the test AUC rises from 0.876 to 0.959, leaving a gap of 0.029 at $k = 20$.

What happens at $k = 1$. With $k = 1$ the only neighbor of a training point is the point itself, so its estimated probability is exactly its own label, $\hat{p}(\mathbf{x}_i) = y_i \in \{0, 1\}$. Every malignant case gets probability 1 and every benign case gets 0, which separates the classes perfectly, so the training ROC curve goes straight to the top-left corner and the training AUC is 1 (assuming no duplicated predictor vectors with different labels). This says nothing about the model’s quality: 1NN has simply memorized the training labels. On the test data the probability is the label of a single nearby training case, which is very noisy and still only 0 or 1, so the test ROC curve has a single corner and its AUC (0.876) is the lowest of all the values of k .

As k increases, each training point's own label carries weight only $1/k$ in its estimate, so the training probabilities stop reproducing the labels and the training AUC drops toward the test AUC, while averaging more neighbors reduces the variance of the test probabilities and the test AUC improves. Training performance is therefore an optimistic and, for small k , badly misleading measure of performance on new observations; model selection (here, the choice of k) should be based on held-out test data or cross-validation instead.

Question 3: Peer evaluation of Homework 5

Reports assigned in [Issue #1](#): [stanlyyp](#) and [rahafala](#).

1. Mistake or item for improvement. I selected [stanlyyp's report](#), **Question 2(a)** (Compare KNN with lasso). The reported test MSE values are far too large to be correct. In the independent setting, KNN is 2.126 and lasso is 2.065, both more than twice the variance of the response itself (about 0.87). In the correlated setting, lasso is 1.934, again above the response variance (about 1.3). These models would therefore do worse than predicting the training mean for every test observation. The data generation or the MSE calculation has an error, and the interpretation in Question 2(b) and 2(c) is based on these numbers.

2. Assessment (written without AI assistance).

The test MSEs in Q2(a) don't look right. Lasso is showing around ~ 2 (2.065 independent, 1.934 correlated), but $\text{Var}(Y)$ is only ~ 0.87 for independent and ~ 1.322 for correlated. Predicting the mean would literally beat these models, and an optimal linear fit should be sitting near ~ 0.25 . Seems like there's a bug in the noise setup, response formula, or the MSE calculation itself.

3. Correction. In the independent setting, $\text{Var}(Y) = 0.25 + \text{Var}(\sin X_2) + 0.09 \text{Var}(X_3^2) + 0.01 \approx 0.25 + 0.43 + 0.18 + 0.01 = 0.87$. A linear model can capture at most $0.5X_1$ and the linear part $E[\cos X_2] X_2 = e^{-1/2} X_2$ of $\sin X_2$, so the lasso's test MSE should be close to the floor $0.87 - 0.25 - e^{-1} \approx 0.25$, not about 2. The code below regenerates both settings as specified (noise standard deviation 0.1, $n = 400$ training and 1,000 test observations, 5NN, and lasso with 10-fold CV over 41 penalties from 10^{-4} to 1, standardizing within each training fold with divisor equal to its size).

```
library(glmnet)
f <- function(X) 0.5 * X[, 1] + sin(X[, 2]) - 0.3 * X[, 3]^2
# Equicorrelated covariates: x = sqrt(1 - rho) z + sqrt(rho) w 1 has unit variances and correlation rho.
gen2 <- function(n, rho) {
  X <- sqrt(1 - rho) * matrix(rnorm(n * 30), n) + sqrt(rho) * rnorm(n)
  list(X = X, y = f(X) + rnorm(n, sd = 0.1))
}
lams <- 10^seq(0, -4, length.out = 41)
lasso_fit <- function(X, y) {
  m <- colMeans(X); s <- sqrt(colMeans(sweep(X, 2, m)^2)); ym <- mean(y)
  tf <- function(Z) sweep(sweep(Z, 2, m), 2, s, "/")
  g <- glmnet(tf(X), y - ym, lambda = lams, standardize = FALSE, intercept = FALSE, thresh = 1e-12)
  list(g = g, pred = function(Z) ym + as.matrix(predict(g, tf(Z))))
}

set.seed(432)
res <- t(sapply(c(0, 0.8), function(rho) {
  tr <- gen2(400, rho); te <- gen2(1000, rho)
  folds <- sample(rep(1:10, length.out = 400))
  cvm <- rowMeans(sapply(1:10, function(v) {
    fit <- lasso_fit(tr[X[folds != v, ], tr$y[folds != v])
    colMeans((fit$pred(tr[X[folds == v, ], drop = FALSE)) - tr$y[folds == v])^2)
  })))
  j <- which.min(cvm); fit <- lasso_fit(tr[X, tr$y])
  c(rho = rho, var_y_test = mean((te$y - mean(te$y))^2),
    mse_5NN = mean((knn_prob(tr[X, tr$y, te[X, 5]) - te$y)^2),
```

```

    mse_lasso = mean((fit$pred(te$X)[, j] - te$y)^2),
    lambda = lams[j], nonzero = sum(coef(fit$g)[-1, j] != 0))
  })
knitr::kable(data.frame(setting = 1:2, res), digits = 4)

```

setting	rho	var_y_test	mse_5NN	mse_lasso	lambda	nonzero
1	0.0	0.8812	0.6020	0.2847	0.0316	4
2	0.8	1.3215	0.1721	0.2319	0.0200	8

`knn_prob` (from the setup chunk) averages the responses of the k nearest training points, so with a numeric response it is equal-weight KNN regression. In both settings every test MSE is well below the response variance in `var_y_test`, so every method beats predicting a constant, as it should. The independent-setting lasso MSE is near the 0.25 floor. The correlated design has a larger response variance because the covariates move together, so its floor differs, but the lasso is again near 0.23, not about 2. The qualitative conclusion in stanlyyp's 2(b), that correlation helps 5NN and lets it overtake the lasso, still holds. However, the MSE values should be recomputed from correctly generated data before they are reported or compared.

GitHub Issue posted: <https://github.com/stanlyyp/stat432-fall2026/issues/3>